Appendix: Source Code (condensed)

Full code: `https://github.com/YOUR-REPO` (`building.py`, `individuals.py`, `scenarios.py`, `simulate.py`, `visualize.py`, `test.py`). Shown below: the model itself. Comments are shortened, data tables are truncated (`# ...`), and run settings are collected in one dictionary `S`. Not shown: `visualize.py` (figures) and `test.py` (sensitivity and robustness tests). Their plotting code was generated with AI assistance from our specifications: which parameters and ranges to test, which scenarios to compare, and what each figure and axis shows.

```
import math, random
from collections import Counter

# ====================== building.py ======================
# The hall as a network: 22 seating spots, 4 stairs, 8 exits.
WALK_SPEED  = 1.61           # m/s, reference speed of the tables
MAX_DENSITY = 3.8            # people/m^2 (SFPE)
STAIR_TIME_LECTURE = 9 / 0.97          # 9.28 s down a 9 m stair
STAIR_TIME_CONCERT = 9 / 0.44          # 20.45 s
SEAT_AREA, AISLE_EXTRA = 7 * 0.0929, 6.1   # m^2 per seat, per spot
SPECIFIC_FLOW = 1.316        # people/s per m of effective width

SPOT_EXITS = {               # floor-1 spot -> its own doors (m)
    "S1": {"A": 17, "B": 30},  "S2": {"A": 18, "B": 26},
    # ... S3-S15 (team measurements)
    "S16": {"A": 9, "B": 23}}
ALL_DOORS = {                # floor-1 spot -> EVERY door (m)
    "S1": {"A": 17, "B": 30, "C": 32, "D": 35, "E": 39,
           "F": 40, "G": 28, "H": 38},
    # ... S2-S16
}
BALCONY_STAIRS = {           # balcony spot -> stair (m)
    "S17": {"X": 8.2}, "S18": {"X": 20.1, "Y": 6.9},
    # ... S19-S22
}
STAIR_LANDS_IN = {           # stair -> landing spot: share
    "X": {"S16": 1.0}, "Y": {"S13": 0.3, "S14": 0.7},
    "Z": {"S12": 0.7, "S13": 0.3}, "W": {"S10": 1.0}}
WALL_ORDER = list("ABCDEFGH")          # doors along the wall
WALL_DIST = {("A", "B"): 19.2, ("B", "C"): 16.4,
             ("C", "D"): 7.7, ("D", "E"): 8.1, ("E", "F"): 18.2,
             ("F", "G"): 16.9, ("G", "H"): 21.2}
SEATS = {"S1": 17, "S2": 72, "S5": 208, "S16": 40}  # ...1,326
EXIT_WIDTH = {e: 1.7 if e in "CDE" else 1.1 for e in WALL_ORDER}
EXIT_LANES = {e: 3 if e in "CDE" else 2 for e in WALL_ORDER}


def door_flow(width, per_m):     # SFPE: 0.15 m lost each side
    return per_m * (width - 0.3)


def crowd_speed(people, area, v_max):   # SFPE speed-density
    v = 1.4 * (1 - 0.266 * people / area)
    return max(0.1, min(v_max, v))


INFO = {}
for e in WALL_ORDER:          # door = `lanes` parallel servers
    f = door_flow(EXIT_WIDTH[e], SPECIFIC_FLOW)  # 1.05 / 1.84 /s
    INFO[e] = {"kind": "exit", "flow": f, "lanes": EXIT_LANES[e],
               "t_pass": EXIT_LANES[e] / f}      # s per person
for x in STAIR_LANDS_IN:      # stair 1.2 m x 9 m: 0.9 /s
    INFO[x] = {"kind": "stair", "flow": door_flow(1.2, 1.0),
               "cap": int(MAX_DENSITY * 1.2 * 9)}
for s, n in SEATS.items():
    area = SEAT_AREA * n + AISLE_EXTRA
    INFO[s] = {"kind": "spot", "flow": None, "area": area,
               "cap": int(MAX_DENSITY * area)}

# edges: walking time (s) at WALK_SPEED
NEXT = {a: {b: m / WALK_SPEED for b, m in d.items()}
        for a, d in {**SPOT_EXITS, **BALCONY_STAIRS}.items()}
for x, shares in STAIR_LANDS_IN.items():
    NEXT[x] = {s: STAIR_TIME_LECTURE for s in shares}
CM_NEXT = {s: {e: m / WALK_SPEED for e, m in d.items()}
           for s, d in ALL_DOORS.items()}   # informed people
# CAMERA_NEXT: own door, then along the wall (all_doors=False)
# TIME_TO_EXIT: the recursion below for an empty hall


def wall_options(door, known_blocked):
    """From a blocked door: nearest open door left and right."""
    options, k = {}, WALL_ORDER.index(door)
    for step in (-1, +1):
        dist, i = 0.0, k
        while 0 <= i + step < len(WALL_ORDER):
            a, b = sorted((WALL_ORDER[i], WALL_ORDER[i + step]))
            dist += WALL_DIST[(a, b)]
            i += step
            if WALL_ORDER[i] not in known_blocked:
                options[WALL_ORDER[i]] = dist
                break
    return options


def dynamic_time_to_exit(queue, scale, stair_time,
                         doors=None, blocked=()):
    """Bellman recursion, recomputed every step: time from each
    node to the outside, incl. the wait Q/f at later bottlenecks.
    Informed people use their own routes and skip blocked doors."""
    def wait(n):
        return queue.get(n, 0) / INFO[n]["flow"]
    h = {e: 0.0 for e in WALL_ORDER}
    for s in SPOT_EXITS:
        opts = doors[s] if doors is not None else NEXT[s]
        h[s] = min((scale * t + wait(e) for e, t in opts.items()
                    if e not in blocked), default=float("inf"))
    for x, w in STAIR_LANDS_IN.items():   # expected over landings
        h[x] = stair_time + sum(p * h[s] for s, p in w.items())
    for b in BALCONY_STAIRS:
        h[b] = min(scale * t + wait(x) + h[x]
                   for x, t in NEXT[b].items())
    return h


# ====================== individuals.py ====================
class Person:
    def __init__(self, pid, node, t_ready):
        self.id, self.node, self.t_ready = pid, node, t_ready
        self.next_node, self.state = None, "waiting"
        self.known_blocked, self.visited = set(), {node}
        self.t_escape, self.t_exit, self.t_queue = 0.0, None, None


def choose_next(node, queue, blocked=(), power=0.2, rng=random,
                choices=None, scale=1.0, h=None):
    """Multinomial logit: P(n) ~ exp(-power * T(n)), with
    T(n) = walk to n + time from n to outside + Q_n / f_n."""
    if choices is None:
        choices = NEXT[node]
    options, times = [], []
    for n, c in choices.items():
        if n in blocked:
            continue
        T = scale * c + (h[n] if h else scale * TIME_TO_EXIT[n])
        if INFO[n]["flow"]:
            T += queue.get(n, 0) / INFO[n]["flow"]   # wait at n
        options.append(n)
        times.append(T)
    if not options:
        return None
    w = [math.exp(-power * (T - min(times))) for T in times]
    return rng.choices(options, weights=w)[0]


# ====================== scenarios.py ======================
REACT_CAP = {"students": 26, "concert": 224, "concert + CM": 168}
AUDIENCES = {   # reaction ~ LogNormal(mu, sigma^2), capped
    "students": {"react_mu": 2.418, "react_sigma": 0.527,
                 "speed": 1.321, "power": 0.08,
                 "stair_time": STAIR_TIME_LECTURE},
    "concert": {"react_mu": 3.358, "react_sigma": 0.497,
                "speed": 1.056, "power": 0.23,
                "stair_time": STAIR_TIME_CONCERT},
    "concert + CM": {"react_mu": 3.070, "react_sigma": 0.497,
                     "speed": 1.056, "power": 0.23,
                     "stair_time": STAIR_TIME_CONCERT,
                     "informed": True, "all_doors": True}}
for a in AUDIENCES:
    AUDIENCES[a].update(react_cap=REACT_CAP[a], dynamic=True)
```

```
EVENTS = {
    "no event": {},
    "wheelchair at D": {"hold": {"D": 8.25}},   # 6 m at 0.727 m/s
    "LED": {"speed_factor": 1.0314},            # walk 3.14% faster
    "cameras": {"informed": True, "all_doors": True},
    "A+B blocked": {"blocked": {"A", "B"}},
    "C+D+E blocked": {"blocked": {"C", "D", "E"}},
    "F+G blocked": {"blocked": {"F", "G"}},
    "H blocked": {"blocked": {"H"}}}
SCENARIOS = {f"{a} | {e}": {**AUDIENCES[a], **EVENTS[e]}
             for a in AUDIENCES for e in EVENTS}    # 3 x 8 = 24
RUNS = 20                                           # seeds each


def mean_and_ci(values):   # mean +- 1.96 sd/sqrt(R), complete runs
    v = [x for x in values if x is not None]
    m = sum(v) / len(v)
    sd = math.sqrt(sum((x - m) ** 2 for x in v) / (len(v) - 1))
    return m, 1.96 * sd / math.sqrt(len(v))


# ====================== simulate.py ======================
DT, T_MAX = 0.25, 700            # time step, stop time (s)


def start_walking(p, queue, occ, rng, S, heading, h, doors):
    """Choose the next node and set the walking timer."""
    scale = WALK_SPEED / (S["speed"] * S["speed_factor"])
    if INFO[p.node]["kind"] == "stair":       # land by shares
        w = STAIR_LANDS_IN[p.node]
        p.next_node = rng.choices(list(w), list(w.values()))[0]
        p.t_escape, p.state = S["stair_time"], "walking"
        return
    if S["informed"] and p.node in doors:     # CM / cameras
        choices = doors[p.node]
        target = choose_next(p.node, heading, p.known_blocked,
                             S["power"], rng, choices, scale)
    else:
        choices = NEXT[p.node]
        target = choose_next(p.node, queue, p.known_blocked,
                             S["power"], rng, scale=scale, h=h)
    if target is None:
        return
    walk = choices[target] * scale
    if INFO[p.node]["kind"] == "spot":        # crowded spot
        v = S["speed_factor"] * crowd_speed(
            occ[p.node], INFO[p.node]["area"], S["speed"])
        walk = choices[target] * WALK_SPEED / v
    p.next_node, p.t_escape, p.state = target, walk, "walking"


def run(seed, settings):
    S = {"informed": False, "all_doors": False, "hold": {},
         "speed_factor": 1.0, "blocked": set(), **settings}
    rng, blocked, hold = random.Random(seed), S["blocked"], S["hold"]
    doors = CM_NEXT if S["all_doors"] else CAMERA_NEXT
    scale = WALK_SPEED / (S["speed"] * S["speed_factor"])
    people = []
    for spot, n in SEATS.items():             # reaction times
        for _ in range(n):
            r = rng.lognormvariate(S["react_mu"], S["react_sigma"])
            if S["react_cap"] is not None:
                r = min(r, S["react_cap"])
            p = Person(len(people), spot, r)
            if S["informed"]:
                p.known_blocked = set(blocked)
            people.append(p)
    occ = Counter(p.node for p in people)
    lane_free = {e: [0.0] * INFO[e]["lanes"] for e in WALL_ORDER}
    stair_free = dict.fromkeys(STAIR_LANDS_IN, 0.0)
    held, t, inside = set(), 0.0, people[:]
    while inside and t < T_MAX:
        queue = Counter(p.next_node for p in inside
                        if p.state == "queuing")
        heading = Counter(p.next_node for p in inside   # seen by
                          if p.state in ("walking", "queuing")
                          and p.next_node in lane_free)  # CM/cams
        info = S["informed"]
        h = dynamic_time_to_exit(heading if info else queue, scale,
                                 S["stair_time"], doors if info
                                 else None, blocked if info else ())
        inside.sort(key=lambda p: p.t_queue if p.state == "queuing"
                    else float("inf"))                  # FIFO
        args = (rng, S, heading, h, doors)
        for p in inside:
            if p.state == "waiting" and t >= p.t_ready:
                start_walking(p, queue, occ, *args)
            elif p.state == "walking":
                p.t_escape -= DT
                if p.t_escape > 0:
                    continue
                if p.next_node is None:
                    start_walking(p, queue, occ, *args)
                    continue
                p.state, p.t_queue = "queuing", t
                if p.next_node in lane_free:      # now at the door
                    occ[p.node] -= 1
                    occ[p.next_node] += 1
                    p.node = p.next_node
            elif p.state == "queuing":
                tgt = p.next_node
                if tgt in blocked:                # found blocked:
                    p.known_blocked.add(tgt)      # walk along wall
                    occ[p.node] -= 1
                    occ[tgt] += 1
                    p.node = tgt
                    wall = {d: m / (S["speed"] * S["speed_factor"])
                            for d, m in wall_options(
                                tgt, p.known_blocked).items()}
                    p.next_node = choose_next(
                        tgt, {}, p.known_blocked,
                        S["power"], rng, wall)
                    p.t_escape = wall.get(p.next_node, 0.0)
                    p.state = "walking"
                elif tgt in lane_free:            # door: lanes
                    if tgt in hold and tgt not in held:  # wheelchair
                        held.add(tgt)
                        lane_free[tgt] = [t + hold[tgt]] * \
                            len(lane_free[tgt])
                    lanes = lane_free[tgt]
                    i = lanes.index(min(lanes))
                    if lanes[i] <= t:
                        start = max(lanes[i], p.t_queue)
                        lanes[i] = start + INFO[tgt]["t_pass"]
                        occ[p.node] -= 1
                        p.state, p.t_exit = "exited", start
                else:                             # stair / landing
                    stair = tgt in stair_free
                    if occ[tgt] < INFO[tgt]["cap"] and (
                            not stair or stair_free[tgt] <= t):
                        if stair:          # one person per 1/f s
                            start = max(stair_free[tgt], p.t_queue,
                                        t - DT)
                            stair_free[tgt] = start + \
                                1 / INFO[tgt]["flow"]
                        occ[p.node] -= 1
                        occ[tgt] += 1
                        p.node = tgt
                        start_walking(p, queue, occ, *args)
                        if stair:
                            p.t_escape -= t - start
        inside = [p for p in inside if p.state != "exited"]
        t += DT
    return people


def summary(people):        # endpoints measured against all N
    N = len(people)
    times = sorted(p.t_exit for p in people if p.t_exit is not None)
    n95 = math.ceil(0.95 * N)
    return {"stuck": N - len(times),
            "t_last": times[-1] if len(times) == N else None,
            "t_95": times[n95 - 1] if len(times) >= n95 else None}
```